



Construcción de software y toma de decisiones (Gpo 501)

Lab 26 Taller de videojuegos

Carlos Arturo Gómez Ayala - A01711027

ITESM Campus Querétaro

Profesor: Enrique Alfonso Calderón Balderas

Co-Titular: Alejandro Fernández Vilchis

Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 19/05/2026

2D Assets lags (imagenes y relación con la IA y diseñadores)

Spritesheets

Para manejar las imágenes y su movimiento

Dwarf Fortress- Ejemplo de juego gigante con objetos “vivos” y código enorme y que se sigue programando.

3D

Se tiene un monito y a largo plazo es más sencillo manejar este modelo que en 2D.

Assets obj

Qué es un *game loop*?

Un **game loop** (bucle de juego) es la estructura que mantiene al juego corriendo. Cada "vuelta" del loop se llama un **frame** y normalmente se ejecuta ~60 veces por segundo. En cada frame se hacen tres cosas: mientras (el juego esté vivo):

1. INPUT → leer qué teclas/botones está presionando el jugador
2. UPDATE → mover objetos, detectar colisiones, actualizar lógica
3. RENDER → dibujar el estado actual en pantalla

Clase 1 — Vertical Slice

Objetivo de la clase: al cerrar, cada quien tiene en localhost:3000 un juego jugable de principio a fin.

Bloque 1 — Setup del proyecto

Crea una carpeta nueva e inicializa el proyecto. Si necesitas refrescar Express o EJS, revisa el Lab 11 — Express y el Lab 12 — EJS.

```
mkdir highway-dodger
cd highway-dodger
npm init -y
npm install express ejs
```

Crea la siguiente estructura de carpetas y archivos:

```
highway-dodger/
├── package.json
├── server.js
├── views/
```

```
|   └─ index.ejs
└─ public/
    └─ style.css
    └─ game.js
```

server.js (versión inicial):

```
const express = require('express');
const path = require('path');

const app = express();
const PORT = 3000;

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));

app.get('/', (req, res) => {
  res.render('index');
});

app.listen(PORT, () => {
  console.log(`Highway Dodger en http://localhost:${PORT}`);
});
```

views/index.ejs:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Highway Dodger</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <main>
    <h1>HIGHWAY DODGER</h1>
    <canvas id="game" width="320" height="480"></canvas>
  </main>
  <script src="/game.js"></script>
</body>
```

```
</html>
```

public/style.css:

```
* { box-sizing: border-box; margin: 0; padding: 0; }
```

```
body {  
  font-family: 'Courier New', monospace;  
  background: #0a0a14;  
  color: #e0e0ff;  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  padding: 20px;  
}
```

```
main {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  gap: 20px;  
}
```

```
h1 {  
  letter-spacing: 4px;  
  color: #00ffaa;  
  text-shadow: 0 0 12px #00ffaa;  
}
```

```
canvas {  
  background: #111122;  
  border: 2px solid #00ffaa;  
  box-shadow: 0 0 24px rgba(0, 255, 170, 0.3);  
  display: block;  
}
```

public/game.js (vacío por ahora, lo llenamos en el siguiente bloque).

Arranca el servidor y verifica que carga la página en terminal de VS:

```
node server.js
```

Abre <http://localhost:3000> — debes ver el título y un canvas oscuro con borde verde.

Dentro de game.js va el código donde se requiere de la variable comentada de canvas y de la que contexto que pertenece a canvas, delimita algunas acciones básicas, en este ejemplo lo limita a lo de 2D.

```
const canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');
const W = canvas.width;
const H = canvas.height;

// Estado del juego
const state = {
  testX: 0
};

function update(dt) {
  // dt es el tiempo (en segundos) desde el último frame
  state.testX += 100 * dt; // se mueve 100 px por segundo
  if (state.testX > W) state.testX = 0;
}

function render() {
  // Limpiar el canvas
  ctx.fillStyle = '#111122';
  ctx.fillRect(0, 0, W, H);

  // Dibujar un rectángulo de prueba
  ctx.fillStyle = '#00ffaa';
  ctx.fillRect(state.testX, H / 2, 40, 40);
}

let lastTime = performance.now();

function loop(now) {
  const dt = (now - lastTime) / 1000; // milisegundos → segundos
  lastTime = now;
```

```
    update(dt);
    render();

    requestAnimationFrame(loop);
}
```

```
requestAnimationFrame(loop);
```

//Maquinas de estado o state machines ayudan a definir los estados del juego en activo porque el gameloop se ejecuta todo el tiempo

Dt= delta time, que depende de la computadora por lo que estandariza el tiempo. Dt es un valor y se puede poner por ejemplo $100 * dt$ en la variable state por ejemplo

Reemplaza el contenido de game.js:

```
const canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');
const W = canvas.width;
const H = canvas.height;
```

// Configuración

```
const LANES = 4;
const LANE_WIDTH = W / LANES;
const CAR_WIDTH = 32;
const CAR_HEIGHT = 56;
```

// Helper: convierte un índice de carril (0..3) a la coordenada X del centro del auto

```
function laneToX(lane) {
    return lane * LANE_WIDTH + (LANE_WIDTH - CAR_WIDTH) / 2;
}
```

```
const player = {
    lane: 1,    // empieza en el segundo carril
    y: H - 80  // cerca del borde inferior
};
```

// --- Input ---

```
window.addEventListener('keydown', (e) => {
    if (e.key === 'ArrowLeft' || e.key === 'a') {
```

```

    player.lane = Math.max(0, player.lane - 1);
  }
  if (e.key === 'ArrowRight' || e.key === 'd') {
    player.lane = Math.min(LANES - 1, player.lane + 1);
  }
});

function update(dt) {
  // Por ahora el auto solo cambia de carril vía input. Nada que hacer aquí.
}

function render() {
  // Fondo
  ctx.fillStyle = '#111122';
  ctx.fillRect(0, 0, W, H);

  // Jugador
  ctx.fillStyle = '#00ffaa';
  ctx.fillRect(laneToX(player.lane), player.y, CAR_WIDTH, CAR_HEIGHT);
}

let lastTime = performance.now();
function loop(now) {
  const dt = (now - lastTime) / 1000;
  lastTime = now;
  update(dt);
  render();
  requestAnimationFrame(loop);
}
requestAnimationFrame(loop);

```

Codigo completo limpio actualizado de game.js

```

const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");
const H = canvas.height;
const W = canvas.width;

// Configuración
const LANES = 4;

```

```

const LANE_WIDTH = W / LANES;
const CAR_WIDTH = 32;
const CAR_HEIGHT = 56;
const ROAD_SPEED = 320; // pixeles por segundo
const DASH_HEIGHT = 28;
const DASH_GAP = 24;

// Convierte un índice de carril (0..3) a la coord X del centro del auto
function laneToX(lane){
    return lane * LANE_WIDTH + (LANE_WIDTH - CAR_WIDTH) / 2;
}

const player = {
    lane: 1, //empieza en el segundo carril, recuerda empieza en 0
    y: H - 80
};

// Estado del juego
const state = {
    testX: 0
};

// Input
window.addEventListener("keydown", (e) => {
    if(e.key === "ArrowLeft" || e.key === "a"){
        player.lane = Math.max(0, player.lane - 1); // mover a la izquierda, sin salir del
        carril 0
    }
    if(e.key === "ArrowRight" || e.key === "d"){
        player.lane = Math.min(LANES - 1, player.lane + 1); // mover a la derecha, sin
        salir del carril 3
    }
    if(e.key === " " && gameOver){
        enemies.length = 0; // eliminar todos los enemigos
        spawnTimer = 0;
        gameOver = false;
        score = 0;
        player.lane = 1; // reiniciar posición del jugador
    }
});

```

```

let roadOffset = 0;
const enemies = [];
let spawnTimer = 0;
const SPAWN_INTERVAL = 0.9; // segundos entre spawns
let gameOver = false;
let score = 0;

function rectsOverlap(ax,ay,aw,ah,bx,by,bw,bh){
    return ax < bx+bw &&
           ax+aw > bx &&
           ay < by+bh &&
           ay+ah > by;
}

function update(dt){
    if(gameOver) return;

    score += dt * 100; // 100 puntos por segundo
    roadOffset = (roadOffset + ROAD_SPEED * dt) % (DASH_HEIGHT +
DASH_GAP);

    //Spawn de rivales
    spawnTimer -= dt;
    if(spawnTimer <= 0){
        spawnTimer = SPAWN_INTERVAL;
        enemies.push({
            lane: Math.floor(Math.random() * LANES),
            y: -CAR_HEIGHT
        });
    }

    //Mover rivales hacia abajo y descartar los que salieron
    for( const e of enemies){
        e.y += ROAD_SPEED * dt;
    }

    // Eliminar enemigos fuera de pantalla
    for(let i = enemies.length -1; i >= 0; i--){
        if(enemies[i].y > H){
            enemies.splice(i, 1);
        }
    }

```

```

    }

    if(!gameOver){
        const px = laneToX(player.lane);
        for(const e of enemies){
            const ex = laneToX(e.lane);
            if(rectsOverlap(px, player.y, CAR_WIDTH, CAR_HEIGHT,
                ex, e.y, CAR_WIDTH, CAR_HEIGHT)){
                gameOver = true;
                break;
            }
        }
    }
}

function render(){
    //Limpiar el canvas
    ctx.fillStyle = "#111122";
    ctx.fillRect(0, 0, W, H);

    // Dibujar un rectángulo de prueba
    //ctx.fillStyle = "#00ffaa";
    //ctx.fillRect(state.testX, H/2, 40, 40);

    // Líneas divisoras entre carriles
    ctx.fillStyle = "#3a3a5a";
    for(let lane = 1; lane < LANES; lane++){
        const x = lane * LANE_WIDTH - 2;
        for(let y = -DASH_HEIGHT + roadOffset; y < H; y += DASH_HEIGHT +
DASH_GAP){
            ctx.fillRect(x,y,4,DASH_HEIGHT);
        }
    }

    // Rivales
    ctx.fillStyle = "ff66cc";
    for(const e of enemies){
        ctx.fillRect(laneToX(e.lane), e.y, CAR_WIDTH, CAR_HEIGHT);
    }

    // Jugador

```

```

    ctx.fillStyle = "#00ffaa";
    ctx.fillRect(laneToX(player.lane), player.y, CAR_WIDTH, CAR_HEIGHT);

    // Score
    ctx.fillStyle = '#00ffaa';
    ctx.font = 'bold 16px "Courier New", monospace';
    ctx.textAlign = 'left';
    ctx.fillText('SCORE: ' + Math.floor(score), 10, 24);

    if (gameOver) {
        ctx.fillStyle = 'rgba(0, 0, 0, 0.7)';
        ctx.fillRect(0, 0, W, H);

        ctx.fillStyle = '#ff66cc';
        ctx.font = 'bold 28px "Courier New", monospace';
        ctx.textAlign = 'center';
        ctx.fillText('GAME OVER', W / 2, H / 2 - 10);

        ctx.fillStyle = '#e0e0ff';
        ctx.font = '14px "Courier New", monospace';
        ctx.fillText('Presiona ESPACIO para reiniciar', W / 2, H / 2 + 20);
    }
}

let lastTime = performance.now();

function loop(now){
    const dt = (now - lastTime) / 1000; // milisegundos a segundos
    lastTime = now;

    update(dt);
    render();

    requestAnimationFrame(loop);
}

requestAnimationFrame(loop);

```

Se hacen ajustes para añadir score, velocidad, blur, sombra, etc.
 Ahora para el servidor ir a

Modifica server.js:

```
const express = require('express');
```

```
const path = require('path');
```

```
const app = express();
```

```
const PORT = 3000;
```

```
app.set('view engine', 'ejs');
```

```
app.set('views', path.join(__dirname, 'views'));
```

```
app.use(express.json()); // necesario para leer JSON del body
```

```
app.use(express.static(path.join(__dirname, 'public')));
```

```
// "BD" en memoria
```

```
let scores = [];
```

```
function topScores() {
```

```
  return [...scores].sort((a, b) => b.score - a.score).slice(0, 10);
```

```
}
```

```
app.get('/', (req, res) => {
```

```
  res.render('index', { scores: topScores() });
```

```
});
```

```
app.get('/api/scores', (req, res) => {
```

```
  res.json(topScores());
```

```
});
```

```
app.post('/api/scores', (req, res) => {
```

```
  const { name, score } = req.body;
```

```
  if (typeof name !== 'string' || typeof score !== 'number') {
```

```
    return res.status(400).json({ error: 'name (string) y score (number) son requeridos' });
```

```
  }
```

```
  scores.push({
```

```
    name: name.slice(0, 12),
```

```
    score: Math.floor(score),
```

```
    date: Date.now()
```

```
  });
```

```
  res.json(topScores());
```

```
});
```

```
app.listen(PORT, () => {
  console.log(`Highway Dodger en http://localhost:${PORT}`);
});
```

Volver a game.js y añadir el score de usuario para hacer submitir

// En update, donde detectas la colisión:

```
if (rectsOverlap(/* ... */)) {
  gameOver = true;
  submitScore();
  break;
}
```

Y se añade una nueva función para submitir ese score después de rectsOverlap

// Nueva función:

```
async function submitScore() {
  if (scoreSubmitted) return;
  scoreSubmitted = true;

  const name = (prompt('GAME OVER\nTu nombre (max 12):', 'ANON') ||
'ANON').trim() || 'ANON';

  try {
    const res = await fetch('/api/scores', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, score: Math.floor(score) })
    });
    const top = await res.json();
    console.log('Nuevo top 10:', top);
  } catch (err) {
    console.error('No se pudo enviar el score', err);
  }
}
```

El anon es para que en caso de que no se submita nombre, se ponga un nombre por default.

Acostumbrarse a dobles validaciones para evitar datos que no queremos en base de datos del servidor y se modifiquen cosas que no, Que se revise el dato

let scoreSubmitted = false; //para evitar multiples envios de score

debajo de letScore = 0

Esto para evitar datos repetidos porque usuario al ver botón solo le va a picar muchas veces y van a haber muchos datos repetidos. Hay que deshabilitar ese botón

Edita views/index.ejs porque esta pidiendo los scores entonces:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Highway Dodger</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <main>
    <h1>HIGHWAY DODGER</h1>
    <canvas id="game" width="320" height="480"></canvas>

    <section id="leaderboard">
      <h2>TOP 10</h2>
      <ol>
        <% scores.forEach(function(s) { %>
          <li>
            <span class="name"><%= s.name %></span>
            <span class="score"><%= s.score %></span>
          </li>
        <% }); %>
        <% if (scores.length === 0) { %>
          <li class="empty">Sin puntajes aún — sé el primero.</li>
        <% } %>
      </ol>
    </section>
  </main>
  <script src="/game.js"></script>
```

```
</body>
```

```
</html>
```

Lo importante es mandar mensajes al cliente para que se quede calmado y podamos resolverlo.

Añade estilos para el leaderboard en style.css debajo del estilo de canvas

```
#leaderboard {  
  width: 320px;  
  background: #14142a;  
  border: 1px solid #2a2a4a;  
  padding: 12px 16px;  
}
```

```
#leaderboard h2 {  
  color: #ff66cc;  
  font-size: 14px;  
  letter-spacing: 3px;  
  margin-bottom: 8px;  
}
```

```
#leaderboard ol {  
  list-style: decimal inside;  
  font-size: 14px;  
}
```

```
#leaderboard li {  
  padding: 4px 0;  
  border-bottom: 1px dashed #2a2a4a;  
  display: flex;  
  justify-content: space-between;  
}
```

```
#leaderboard li.empty {  
  color: #666;  
  font-style: italic;  
  justify-content: flex-start;  
}
```

```
#leaderboard .score {
```

```
color: #00ffaa;  
}
```

Refresca la página: ahora ves el top 10 server-side cada vez que entras. Si juegas y mueres, el score se envía vía fetch — pero el render del leaderboard solo se actualiza al refrescar la página entera. Esto es intencional: te muestra **dos formas distintas** de obtener datos:

- **Server-side render con EJS:** datos frescos cuando se carga la página.
- **Fetch desde el cliente:** datos frescos sin recargar.

Reto opcional: después de enviar el score, llama también `await fetch('/api/scores')` y reconstruye el leaderboard en el DOM con JavaScript. Eso sería 100% AJAX, sin recargar.

Por último se pidió una funcionalidad extra para este laboratorio

La funcionalidad que agregué al juego es una pausa activada con la tecla P. Al presionarla durante la partida el juego se detiene completamente y los autos dejan de moverse, el score deja de subir y aparece un mensaje en pantalla indicando que está pausado. Presionando P de nuevo el juego continúa desde donde se quedó. Si ya hay Game Over la pausa no hace nada. Para implementarla agregué una variable `paused`, un event listener para detectar la tecla, una validación al inicio de `update` para no procesar nada mientras está pausado y el render del mensaje en pantalla.

